

Institut Teknologi Bandung
Sekolah Teknik Elektro dan Informatika
Teknik Informatika

Laporan Tugas Besar
Milestone 4: Intermediate Code Generator

GTFOTBFO (HBB)

Nathanael Imandatua Manurung - 13524021
Nicholas Wise Saragih Sumbayak - 13524037
Kurt Mikhael Purba - 13524065
Rava Khoman Tuah Saragih - 13524107

Semester II / 2025–2026

Laboratorium Ilmu dan Rekayasa Komputasi

Daftar Isi

1 Ringkasan Pengerjaan Milestone 4

Milestone 4 merupakan tahap akhir kompilasi bahasa Pascal sederhana, yaitu pembangkitan kode perantara (intermediate code) dan eksekusinya menggunakan interpreter stack-based. Kelompok kami mengimplementasikan generator kode P-Code untuk AST yang dibangun pada Milestone 3, dan interpreter yang mengeksekusi instruksi P-Code tersebut. Seluruh pipeline lengkapnya adalah:

1. **Lexical Analysis** (Scanner) — tokenisasi sumber Pascal
2. **Syntax Analysis** (Parser) — pembangunan Parse Tree
3. **Semantic Analysis** (SemanticAnalyzer + SymbolTableManager) — pemeriksaan tipe dan tabel simbol
4. **Intermediate Code Generation** (CodeGenerator) — pembangkitan instruksi P-Code
5. **Execution** (Interpreter + StackMachine) — eksekusi P-Code

1.1 Struktur File Milestone 4

Berikut file-file baru dan modifikasi yang dilakukan pada milestone ini:

Path	Peran	Deskripsi Singkat
src/intermediate/CodeGenerator.hpp	Role 4	Header CodeGenerator
src/intermediate/CodeGenerator.cpp	Role 4	Visitor AST untuk emit P-Code
src/intermediate/Instruction.hpp	Role 4	Definisi kelas Instruction
src/intermediate/Instruction.cpp	Role 4	Implementasi Instruction::toString()
src/runtime/StackMachine.hpp	Role 3	Header mesin stack
src/runtime/StackMachine.cpp	Role 3	Operasi stack dan frame
src/runtime/Interpreter.hpp	Role 3	Header interpreter P-Code
src/runtime/Interpreter.cpp	Role 3	Implementasi interpreter
src/driver_m4.cpp	Role 4	Driver end-to-end M4

Tabel 1: File-file kunci Milestone 4

2 Implementasi Intermediate Code Generator

2.1 Kelas Instruction

Setiap instruksi P-Code direpresentasikan dengan tiga field: **f** (fungsi), **l** (level), dan **a** (argumen). Fungsi-fungsi yang diimplementasikan adalah:

Kode	Nama	Deskripsi
0	LIT	Push literal a ke stack
1	OPR	Operasi aritmatika/perbandingan (dikodekan di a)
2	LOD	Load variabel (level l , alamat a)
3	STO	Store ke variabel (level l , alamat a)
4	CAL	Panggil prosedur (level l , alamat a)
5	INT	Alokasi frame (level l , ukuran a)
6	JMP	Loncat tak-bersyarat ke alamat a
7	JPC	Loncat ke a jika top-of-stack = 0
8	RET	Return dari prosedur/fungsi

Tabel 2: Instruksi P-Code yang diimplementasikan

2.2 CodeGenerator sebagai AST Visitor

`CodeGenerator` melakukan traversal AST secara rekursif dan mengisi vektor instruksi. Strategi alokasi alamat yang digunakan:

- **Parameter:** mendapat alamat negatif, mulai dari `-psze` sampai `-1`, dihitung dari kiri ke kanan.
- **Variabel lokal:** mendapat alamat positif mulai dari `3` (setelah frame header SL, DL, RA) sampai `3 + vsze - 1`.
- **Konstanta:** di-inline sebagai LIT, tidak dialokasikan di memori.

Untuk memastikan eksekusi tidak jatuh ke tubuh prosedur, `visit(ProgramNode)` memancarkan `JMP` di awal program yang melewati seluruh deklarasi subprogram menuju badan utama program.

3 Implementasi Interpreter dan Stack Machine

3.1 Stack Machine

`StackMachine` mengelola stack eksekusi dan frame aktivasi. Setiap frame memiliki:

- `bp` (base pointer) — awal frame
- `sp` (stack pointer) — elemen teratas stack
- `ip` (instruction pointer) — indeks instruksi berikutnya
- `Display` — array alamat frame untuk tiap level leksikal

Fungsi `pushFrame` mendorong `SL`, `DL`, `RA`, lalu `frameSize - 3` elemen nol. Argumen yang didorong pemanggil sebelum `CAL` berada tepat di bawah base pointer frame baru, sehingga parameter diakses dengan offset negatif.

3.2 Interpreter

`Interpreter` mengeksekusi instruksi satu per satu menggunakan dispatcher `switch` pada field `f`. Penanganan instruksi kunci:

- `handleCAL`: menyimpan `RA = ip + 1`, membuat frame baru, dan melompat ke alamat target.
- `handleINT`: mengalokasikan ruang frame (menambah `sp`) dan memperbarui `display[level]`.
- `handleRET`: memulihkan `ip`, `bp`, dan `display` dari frame saat ini.

Error handling mencakup: stack underflow/overflow, division by zero, integer overflow, invalid jump, dan out-of-bounds memory access.

4 Pengujian

Berikut hasil pengujian untuk lima kasus uji wajib. Semua pengujian berhasil menghasilkan output yang sesuai.

4.1 Kasus 1: Program Dasar (pos_basic.txt)

Program paling sederhana: inisialisasi variabel, lalu cetak nilainya.

```
1 program Basic;  
2 var x: integer;  
3 begin  
4     x := 42;  
5     writeln(x);  
6 end.
```

Output:

```
1 42
```

4.2 Kasus 2: Aritmatika (pos_arith.txt)

Menguji urutan operasi: perkalian lebih tinggi prioritas dari penjumlahan.

```
1 program Arith;  
2 var x: integer;  
3 begin  
4     x := 10 + 20 * 3;  
5     writeln(x);  
6 end.
```

Output:

```
1 70
```

4.3 Kasus 3: If-Else (pos_if_else.txt)

Percabangan dengan kondisi >.

```
1 program IfElse;  
2 var x, y: integer;  
3 begin  
4     x := 10;  
5     if x > 5 then  
6         y := 1  
7     else  
8         y := 0;  
9     writeln(y);  
10 end.
```

Output:

```
1 1
```

4.4 Kasus 4: While Loop (pos_while.txt)

Perulangan while dengan pencetakan nilai menurun.

```
1 program WhileLoop;  
2 var i: integer;  
3 begin  
4     i := 5;  
5     while i > 0 do  
6         begin  
7             writeln(i);  
8             i := i - 1;  
9         end;  
10 end.
```

Output:

```
1 5  
2 4  
3 3  
4 2  
5 1
```

4.5 Kasus 5: For Loop (pos_for.txt)

Perulangan for dengan pencetakan nilai 1 sampai 5.

```
1 program ForLoop;  
2 var i: integer;  
3 begin  
4     for i := 1 to 5 do  
5         begin  
6             writeln(i);  
7         end;  
8 end.
```

Output:

```
1 1  
2 2  
3 3  
4 4  
5 5
```

4.6 Kasus Tambahan: Prosedur dengan Parameter

Menguji pemanggilan prosedur dengan satu parameter integer.

```
1 program ProcParam;  
2  
3 var x: integer;  
4  
5 procedure printNum(n: integer);
```

```
6  begin
7      writeln(n);
8  end;
9
10 begin
11     x := 42;
12     printNum(x);
13 end.
```

Output:

```
1  42
```


5 Kesimpulan

Milestone 4 berhasil mengimplementasikan seluruh rantai kompilasi dari analisis leksikal hingga eksekusi kode perantara. Pemecahan masalah parameter passing menjadi hasil penting: dengan menggunakan alamat negatif untuk parameter (karena argumen sudah berada di stack sebelum `CAL`) dan memancarkan `INT` hanya untuk variabel lokal, pemanggilan prosedur dan fungsi dapat berjalan dengan benar. Semua lima kasus uji wajib serta kasus tambahan berhasil dilalui tanpa regresi pada fitur-fitur sebelumnya.